

UIT2734
IOT ARCHITECTURES AND
APPLICATIONS

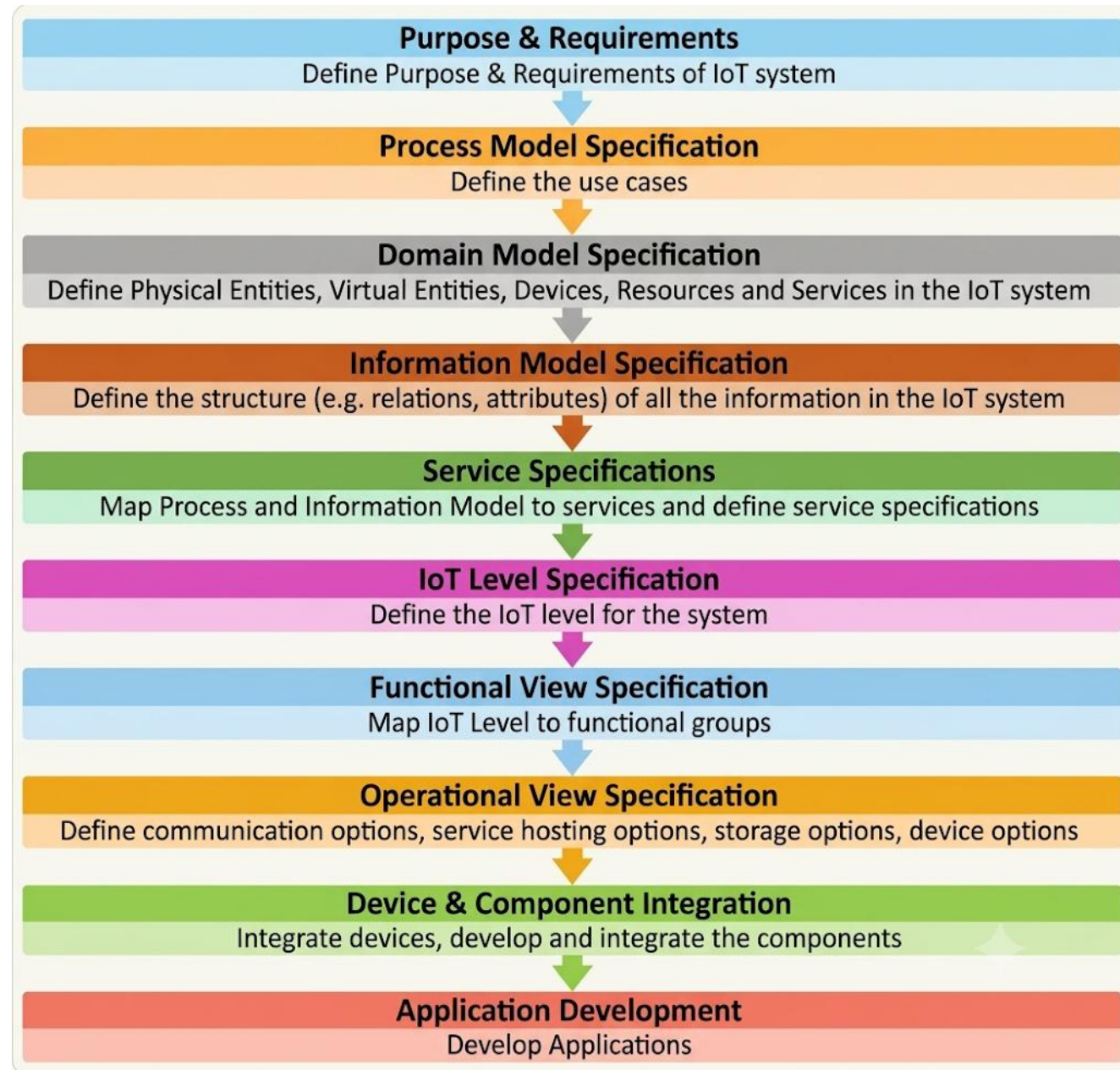
Unit III: Design and Development

Prepared by:

Dr. Mahabub Hasan Mahalat
Assistant Professor
Department of Information Technology
SSN College of Engineering, Chennai

IoT Design Methodology

- IoT systems involve complex interactions between devices, network resources, web services, analytics, application servers, and databases
- A structured methodology ensures that IoT systems are built with clear requirements, proper domain understanding, and well-defined services
- **10-Step IoT Design Methodology**



Purpose & Specification

- First step is to define the purpose and requirements of the system. In this step, the system purpose, behaviour, and requirements are captured.

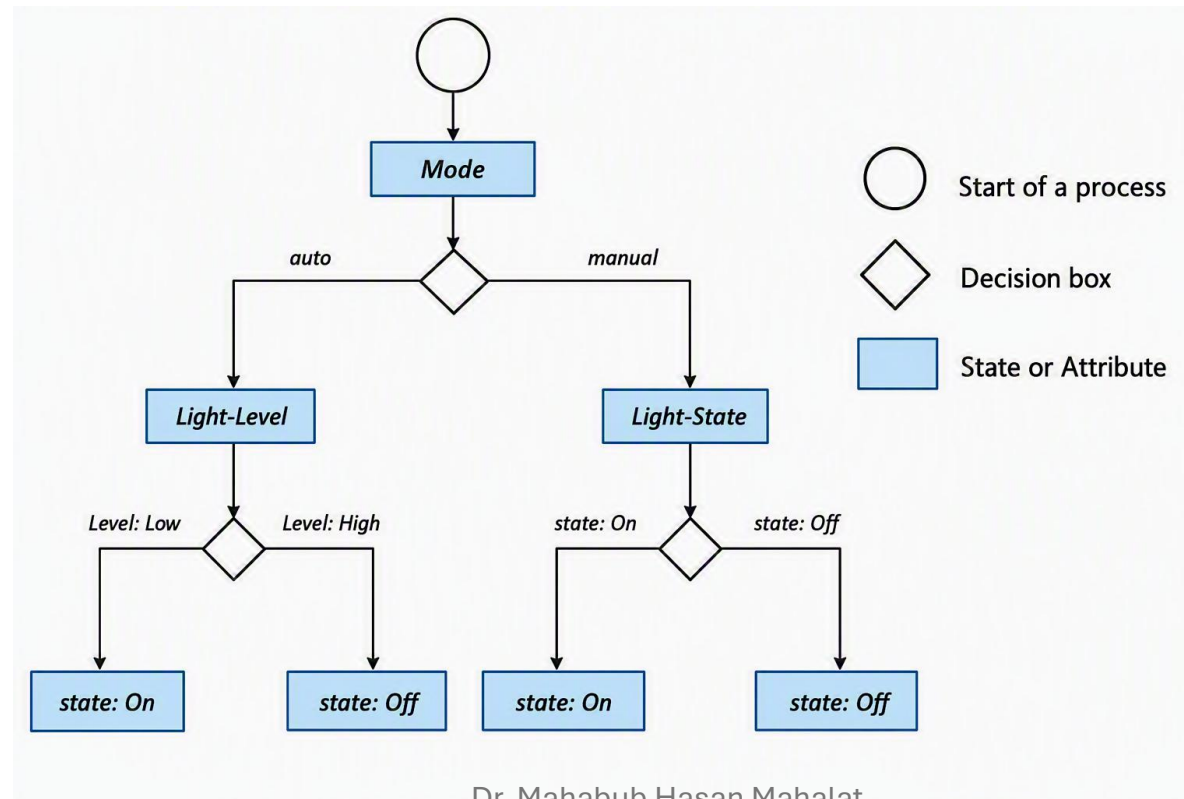
Requirements can be:

- Data collection requirements
- Data analysis requirements
- System management requirements
- Security requirements
- User interface requirements

Case Study – Home Automation System	
Aspect	Specification
Purpose	A home automation system that allows controlling the lights remotely using a web application
Behavior	Supports two modes – Auto and Manual • Auto: Measures light level and switches on when dark • Manual: Allows remotely switching lights on and off
System Management	Provide remote monitoring and control functions
Data Analysis	Perform local analysis of the data
Application Deployment	Deployed locally, but should be accessible remotely
Security	Provide basic security like user authentication

Process Model Specification

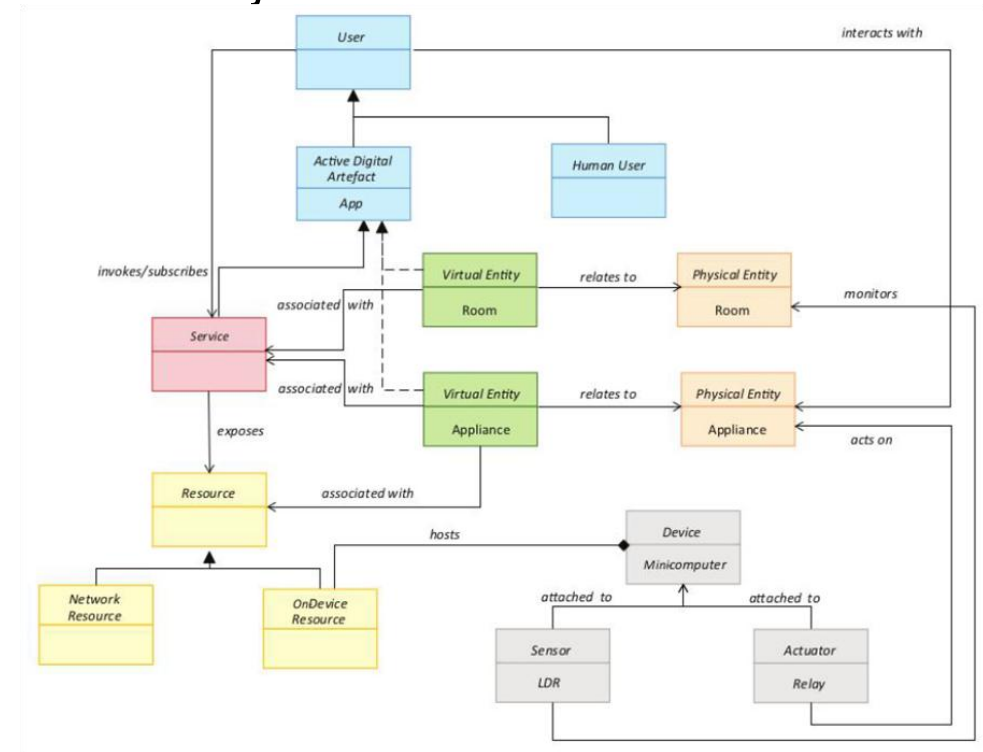
- The use cases of the IoT system are **formally described** based on or derived from the purpose and requirements specifications



Domain Model Specification

- The domain model describes the **main concepts, entities, and objects** in the domain of the IoT system to be designed.
- It defines the attributes of the objects and relationships between objects

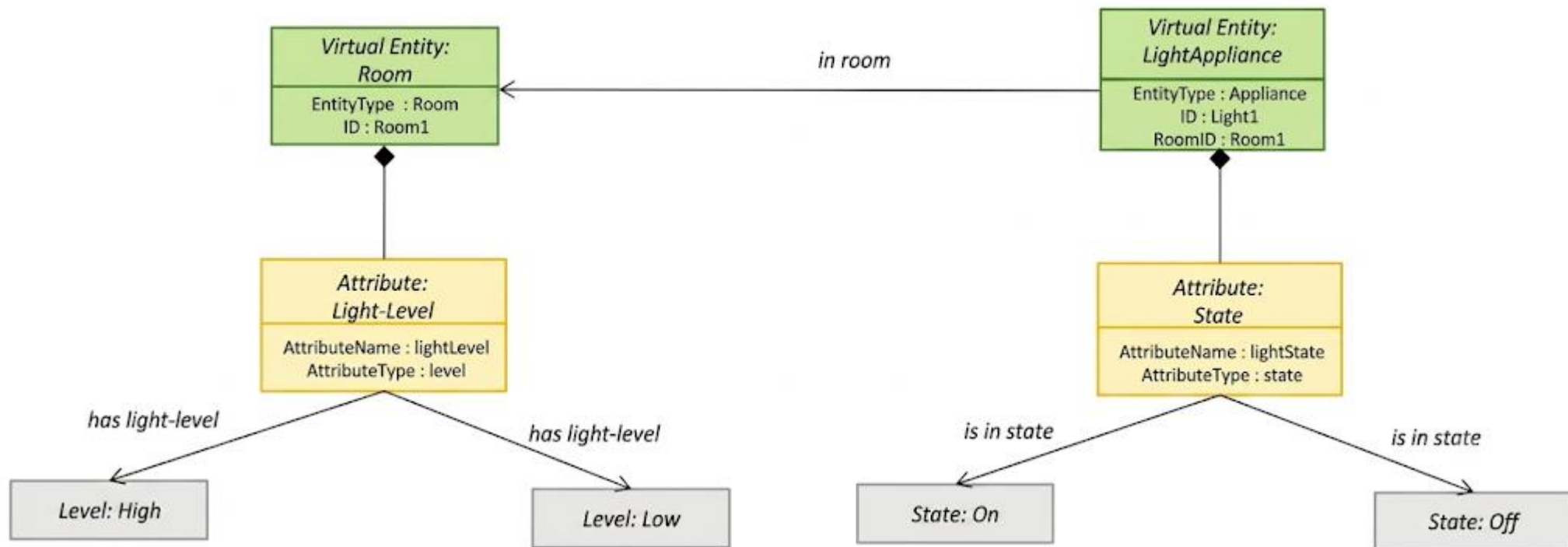
Key Concepts in the Domain Model	
Concept	Description
Physical Entity	Identifiable physical objects monitored or controlled by IoT.
Virtual Entity	Digital representation of a physical entity.
Device	Enables interaction between physical and virtual entities; senses or actuates.
Resource	Software components available on-device or over the network.
Service	Interface for interacting with physical entities using resources.



Domain model specification diagram for home automation

Information Model Specification

- The Information Model does **not** describe how information is represented or stored .
- It defines *what* information exists and its structure.



Service Specification

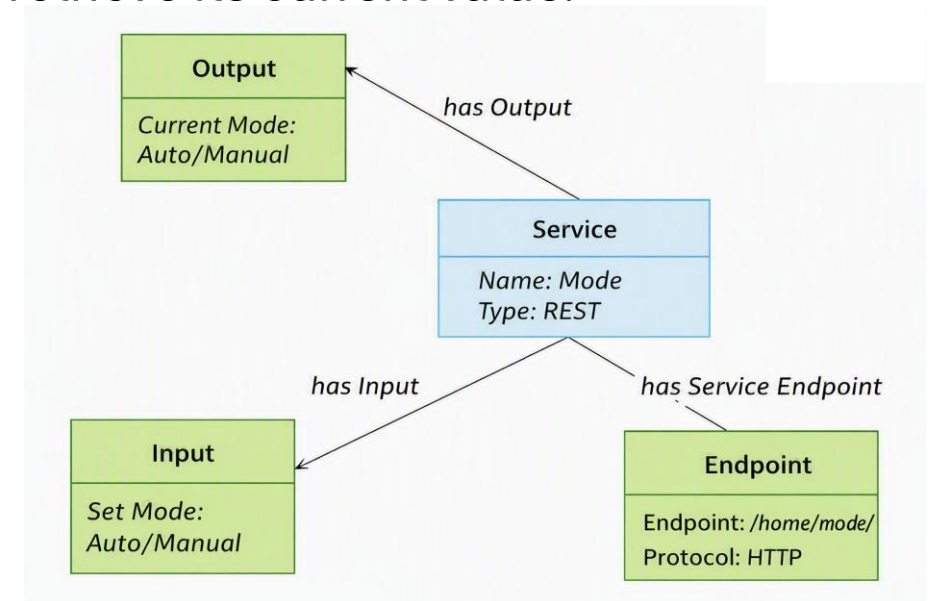
➤ These define all the services in the IoT system.

➤ **How to Create Services:**

Based on the Process Specification (Step 2) and Information Model (Step 4), identify system states and attributes.

➤ For each, define a service to *change* a state/attribute or to *retrieve* its current value.

Element	Description
Service Types	What types of services are provided.
Inputs/Outputs	The data the service requires and returns.
Endpoints	Where the service is accessed.
Schedules	When the service is available.
Preconditions/Effects	Conditions before and after service execution.



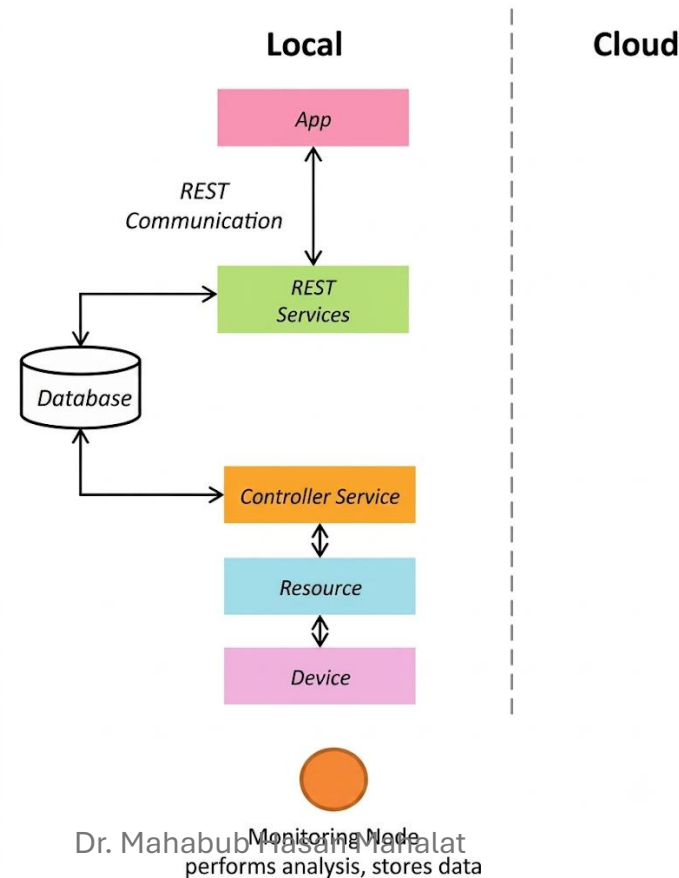
Case Study Example (Home Automation)

Mode Service: Sets the mode to Auto or Manual, or retrieves the current mode.

State Service: Sets the light state to On or Off, or retrieves the current state.

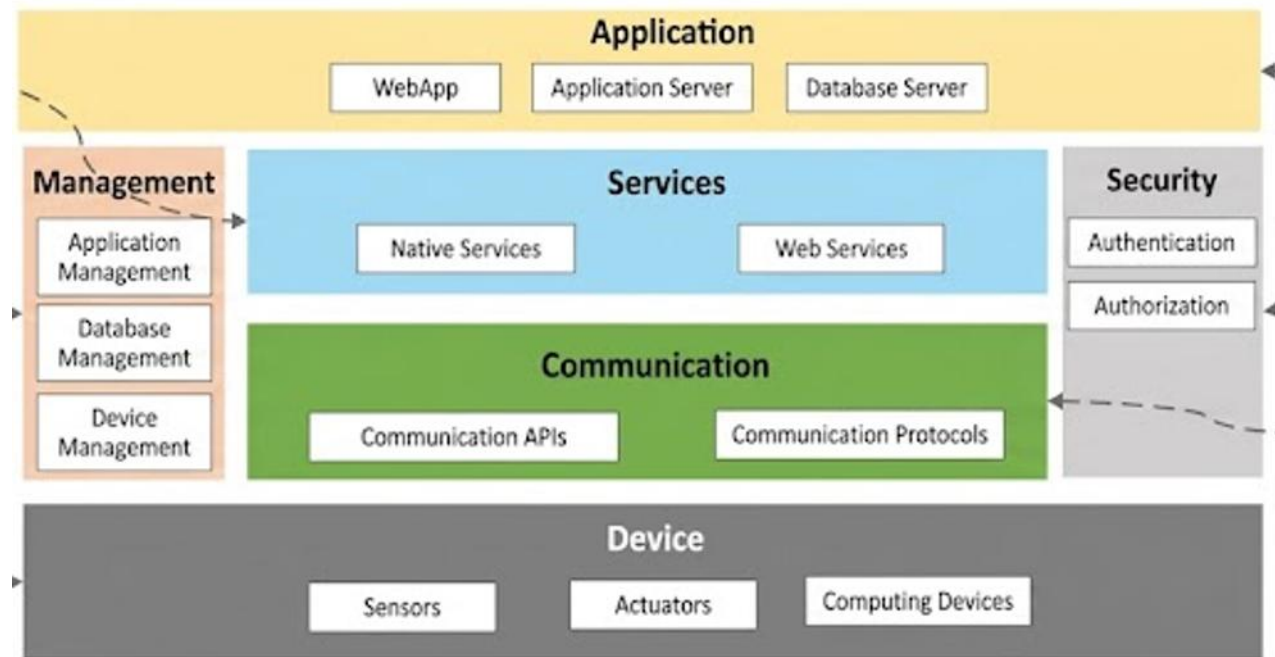
IoT Level Specification

- Defines the appropriate IoT Level for the system.
- This determines the deployment model based on the system's needs.



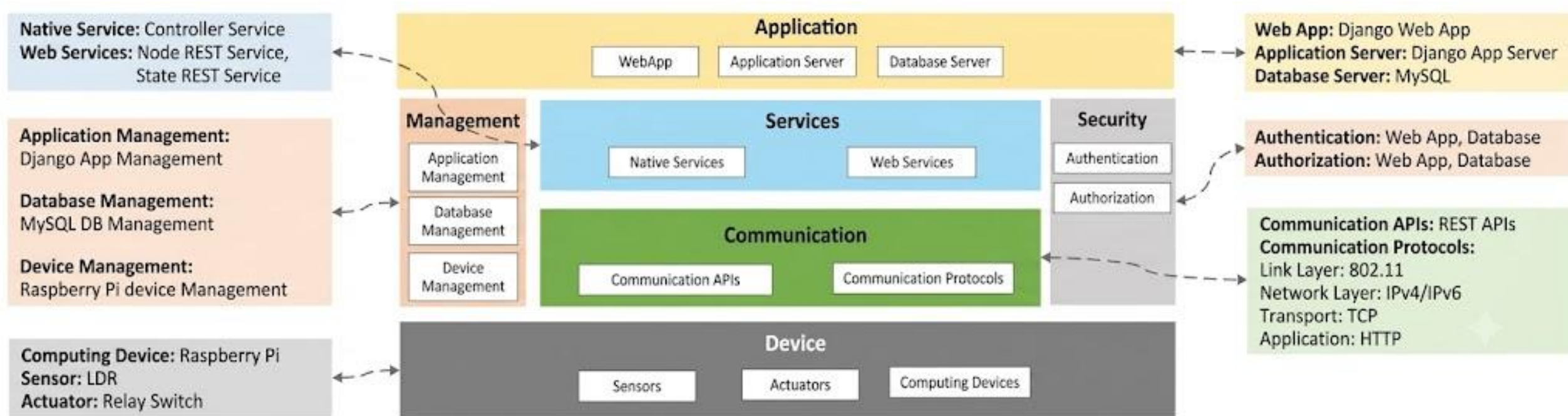
Functional View Specification

- This involves identifying the key functional groups and their interactions.



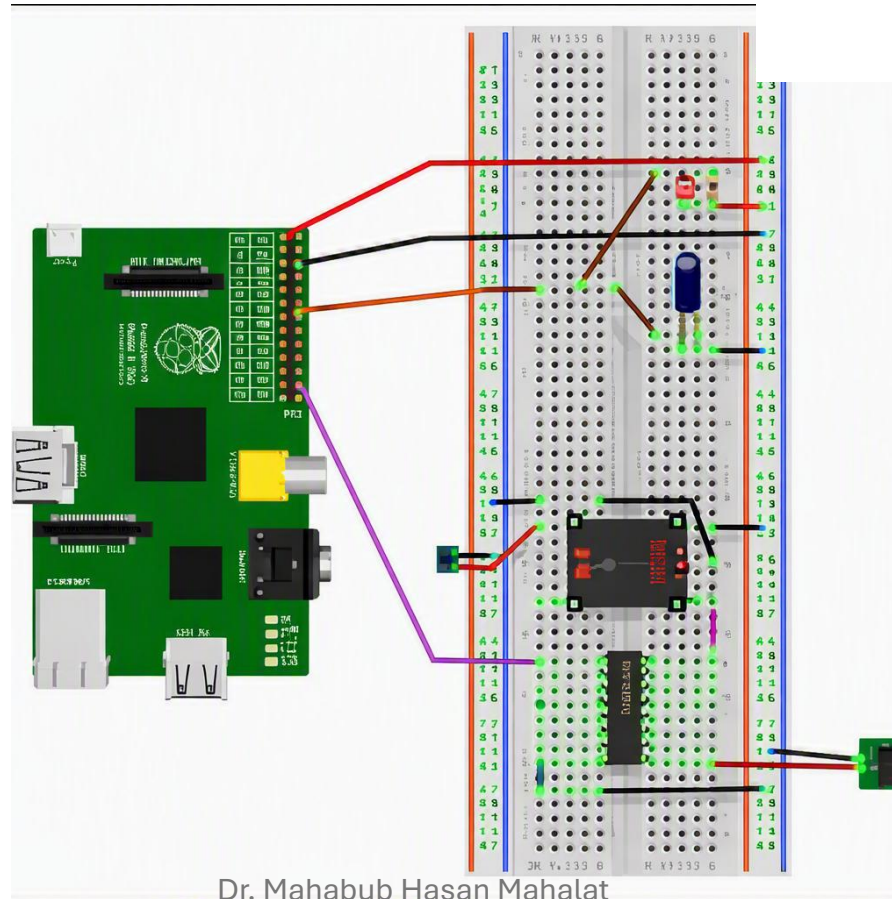
Operational View Specification

- It details how to deploy and operate the system, including hosting, connectivity, and service resolution



Device & Component Integration

- The actual integration of devices and components identified in earlier steps



Application Development

- This is the full development of the IoT application that enables users to interact with the system and utilize its data
- **Case Study Example (Home Automation):**
Development of the web application used for remote monitoring and control.



Embedded Computing Logic

- An embedded device system is a unique computing system that generally runs as a **single application**.
- These systems may or may not connect to the Internet, but in IoT, they connect through internet connections and communicate with other network devices
- Unlike general-purpose computers, embedded control systems are typically designed to perform **specific tasks** within tight cost and development timeline constraints

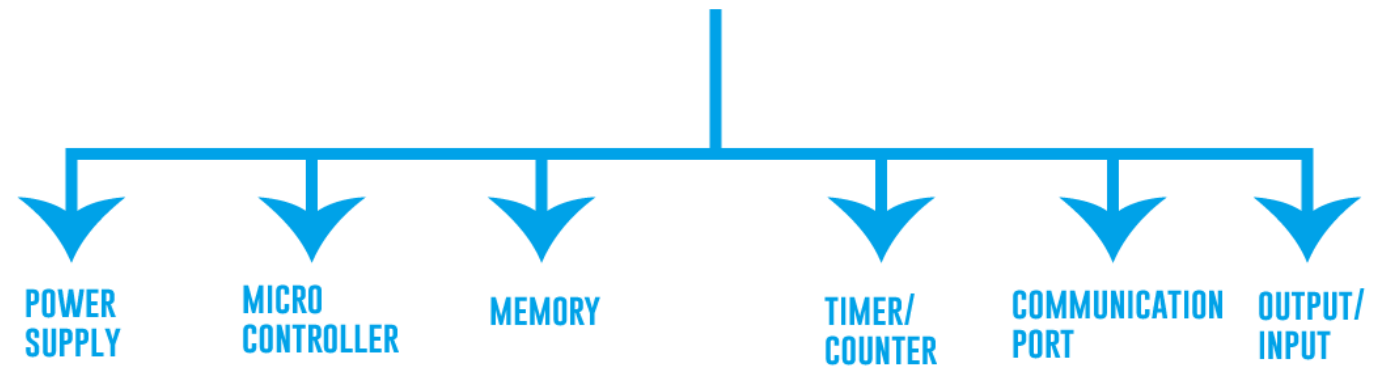
- **Embedded Systems: Historical Context**

Period	Development
1960s	First developed for aerospace and military applications
Today	Control everyday devices: digital cameras, automobiles, smart watches, home appliances, smart garments
Current Industries	Consumer, industrial, automotive, medical, commercial, and military applications

Hardware Components of Embedded Systems

- The selection of a microcontroller and its characteristics—including data processing capabilities, speed, peripherals, and power consumption—is one of the earliest and most critical aspects of (Embedded) IoT system design

EMBEDDED SYSTEM HARDWARE COMPONENTS



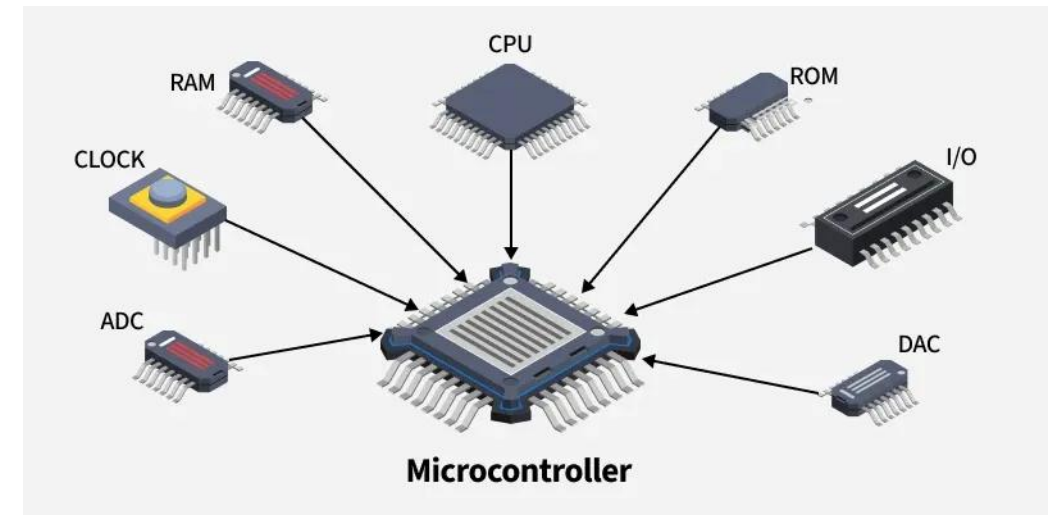
Component	Description
Processor (CPU)	Executes instructions and performs computations
Memory (RAM/Flash)	Stores program code and data
I/O Interfaces	Connects to sensors, actuators, and other devices
Timers/Counters	Handles timing and scheduling tasks
Communication Modules	Enables connectivity (UART, SPI, I2C, etc.)

Microcontroller (MCU)

- A Microcontroller (MCU) is a single integrated circuit containing a processor core, memory, and programmable I/O peripherals
- Key Characteristics of MCU:

Feature	Description
Integration	CPU + Memory + I/O on a single chip
Power Consumption	Very low – ideal for battery-powered devices
Cost	Low cost – suitable for mass IoT deployment
Real-Time Response	Deterministic – suitable for control applications
Examples	8051, ARM Cortex-M, ATmega328P (Arduino), ESP32, MSP432

Dr. Mahabub Hasan Mahalat



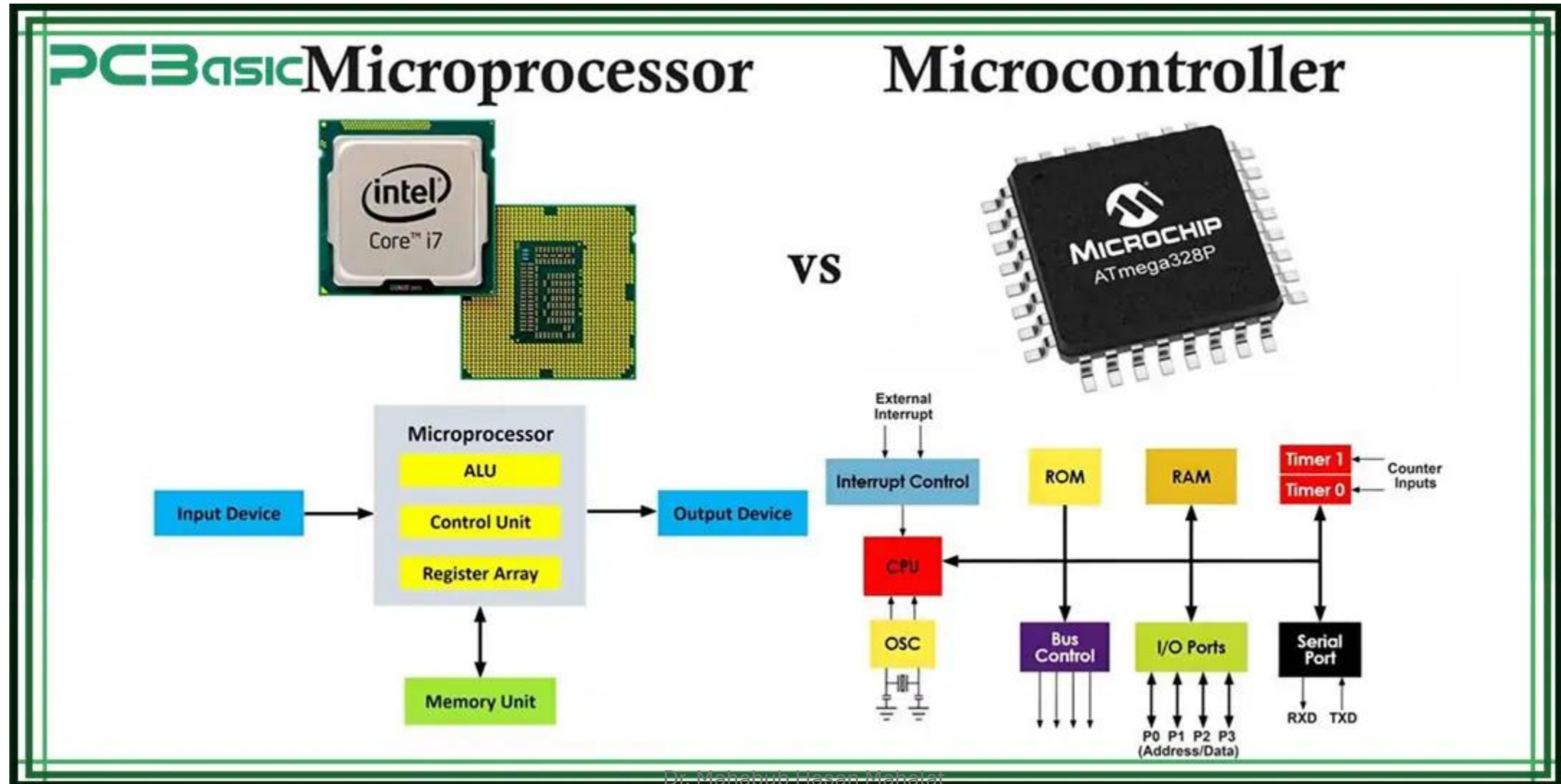
Popular MCU Families for IoT

MCU Family	Key Features	Common Use
8051	8-bit, simple architecture, low cost	Basic embedded applications
ARM Cortex-M	32-bit, low power, rich ecosystem	Professional IoT designs
ATmega328P	8-bit, 16 MHz, Arduino Uno	Prototyping and education
ESP32	32-bit, dual-core, Wi-Fi + BLE	Advanced IoT, edge AI
CC13x2/CC26x2	Ultra-low power, wireless MCU	Battery-powered IoT applications

Microprocessor vs Microcontroller

Microprocessor (MPU)	Microcontroller (MCU)
CPU only on a single chip	CPU + memory + peripherals on a single chip
Requires external RAM, ROM, I/O, timers, etc.	Built-in RAM, Flash/ROM, I/O, timers, ADC, etc.
Designed for general-purpose computing	Designed for dedicated/embedded applications
Higher processing capability	Optimized for control-oriented tasks
Generally higher power consumption	Generally lower power consumption
More external components → larger system	Fewer external components → compact system
Higher overall system cost	Lower system cost
Examples: Intel Core, AMD Ryzen, ARM Cortex-A	Examples: 8051, ATmega328P, STM32, ESP32

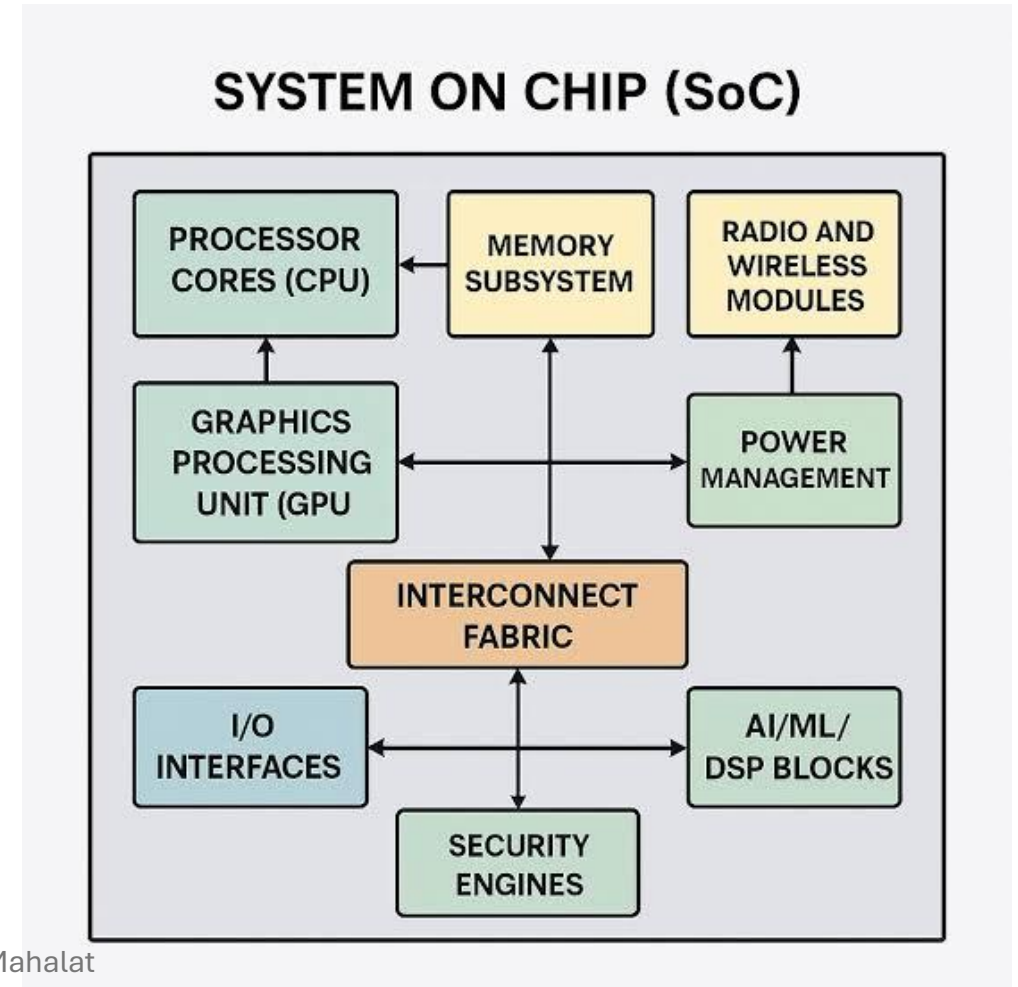
Microprocessor vs Microcontroller



System on Chip (SoC)

- System on Chip (SoC) integrates multiple components of a computer or electronic system onto a **single chip** – including processor, memory, I/O, and often wireless connectivity
- **SoC Key Components:**

Component	Function
Application Processor	Runs complex OS and applications
Real-Time Microcontroller	Handles time-critical tasks (in hybrid SoCs)
Memory	RAM, ROM, cache
Wireless Radios	Wi-Fi, Bluetooth, LoRa, Zigbee, Thread
Analog Interfaces	ADC, DAC, temperature sensors
Security Features	Secure boot, flash encryption, crypto accelerator

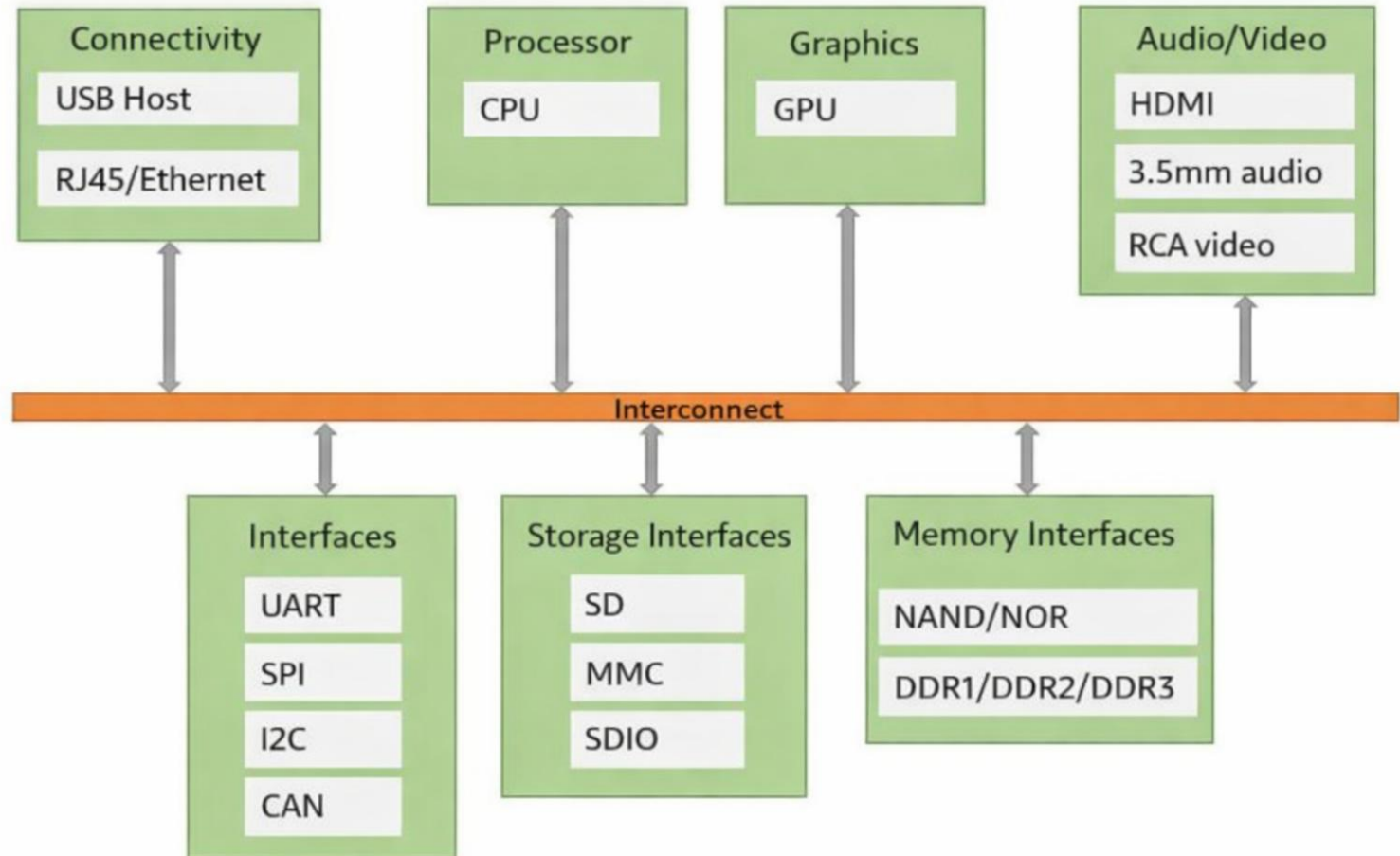


MCU vs. SoC

Aspect	Microcontroller (MCU)	System on Chip (SoC)
Integration	CPU + Memory + I/O	CPU + Memory + I/O + Wireless + GPU + Security
Complexity	Simple, dedicated tasks	Complex, feature-rich applications
Power	Very low	Moderate to low
Connectivity	External module needed	Often built-in (Wi-Fi, BLE, Zigbee, Thread)
Examples	ATmega328P, 8051, Cortex-M	ESP32, Raspberry Pi SoC, Nordic nRF52

- The choice between MCU and SoC depends on application complexity and connectivity needs
- MCUs for simple sensors, SoCs for advanced connected devices with built-in wireless and security features!

Block Diagram of IoT



IoT Devices: Raspberry Pi

- **Raspberry Pi** is a low-cost, credit-card-sized single-board computer developed by the Raspberry Pi Foundation.
- **Key Features:**
 - ARM-based processor
 - RAM (1GB–8GB depending on model)
 - 40-pin GPIO header
 - HDMI output
 - USB ports
 - Ethernet / Wi-Fi / Bluetooth
 - microSD card for storage
- **Operating System:**
 - Runs Linux-based OS (e.g., Raspberry Pi OS)

Components of Raspberry Pi

Components of Raspberry Pi

1. Processor (CPU)

- ARM-based Broadcom SoC
- Acts as the brain of the system
- Executes OS and user programs
- Handles computation and control

2. RAM (Memory)

- Available in 1GB / 2GB / 4GB / 8GB variants
- Stores running programs and OS data
- More RAM → better multitasking

Components of Raspberry PI...

3. microSD Card Slot

- Used as primary storage device
- Stores Operating System and user files
- Functions like a hard disk

4. GPIO Header (40 Pins)

- General Purpose Input/Output pins
- Used to interface sensors and actuators
- Supports:
 - Digital I/O
 - PWM
 - I2C
 - SPI
 - UART



Components of Raspberry Pi...

- **5. USB Ports**

- Connect keyboard, mouse, flash drive
- USB 2.0 / USB 3.0 (model dependent)

- **6. HDMI Port**

- Connects to monitor/display
- Provides audio and video output

- **7. Ethernet Port**

- Wired internet connection
- Supports networking and IoT communication

- **8. Wi-Fi & Bluetooth Module**

- Wireless connectivity
- Used in IoT and remote applications

- **9. Power Port**

- USB-C / Micro-USB (model dependent)
- Typically 5V power supply

- **10. CSI & DSI Ports**

- CSI: Camera Serial Interface (for Pi Camera)
- DSI: Display Serial Interface (for official touchscreen)

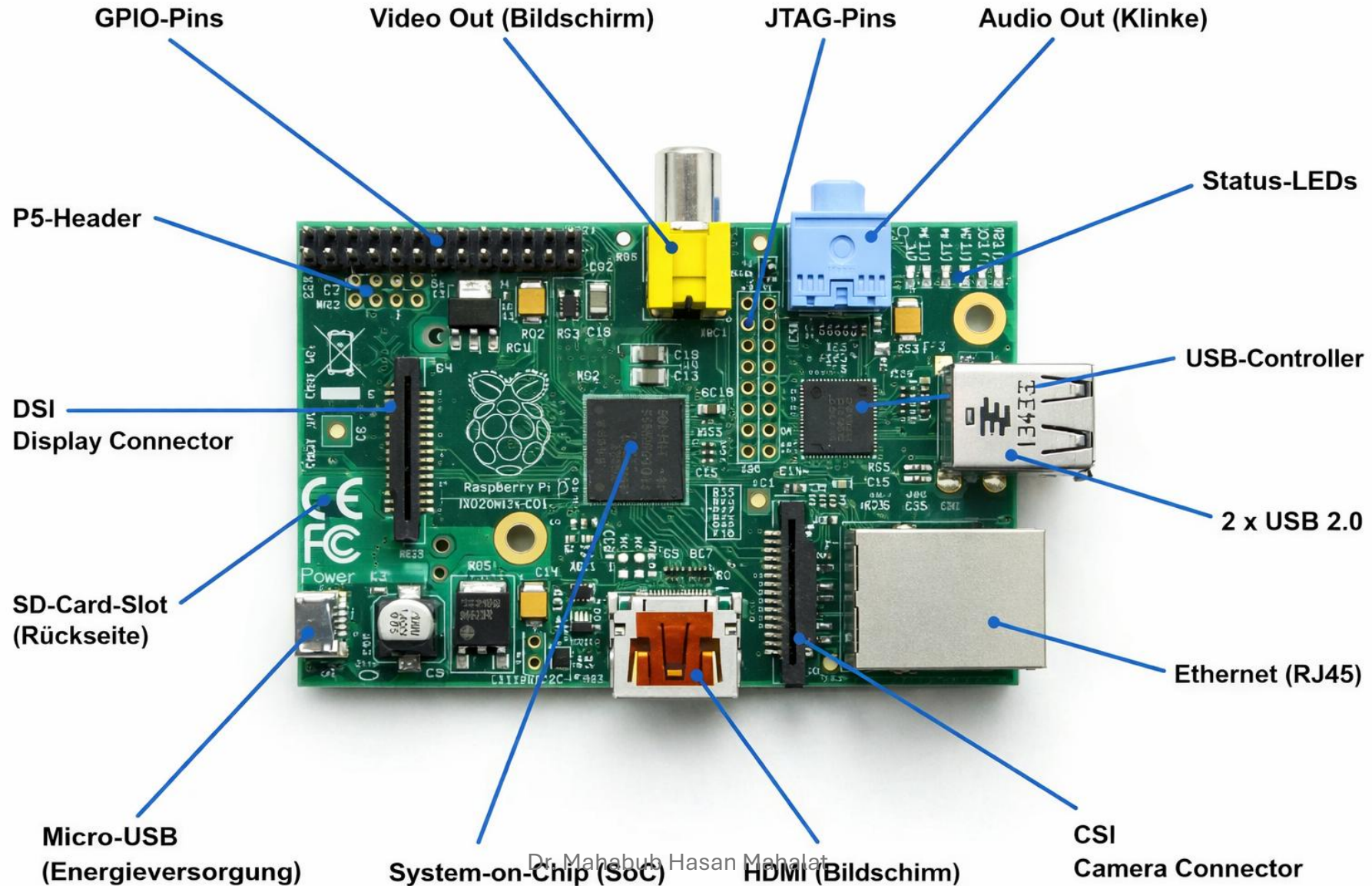
- **11. Audio Jack**

- 3.5mm jack for audio output

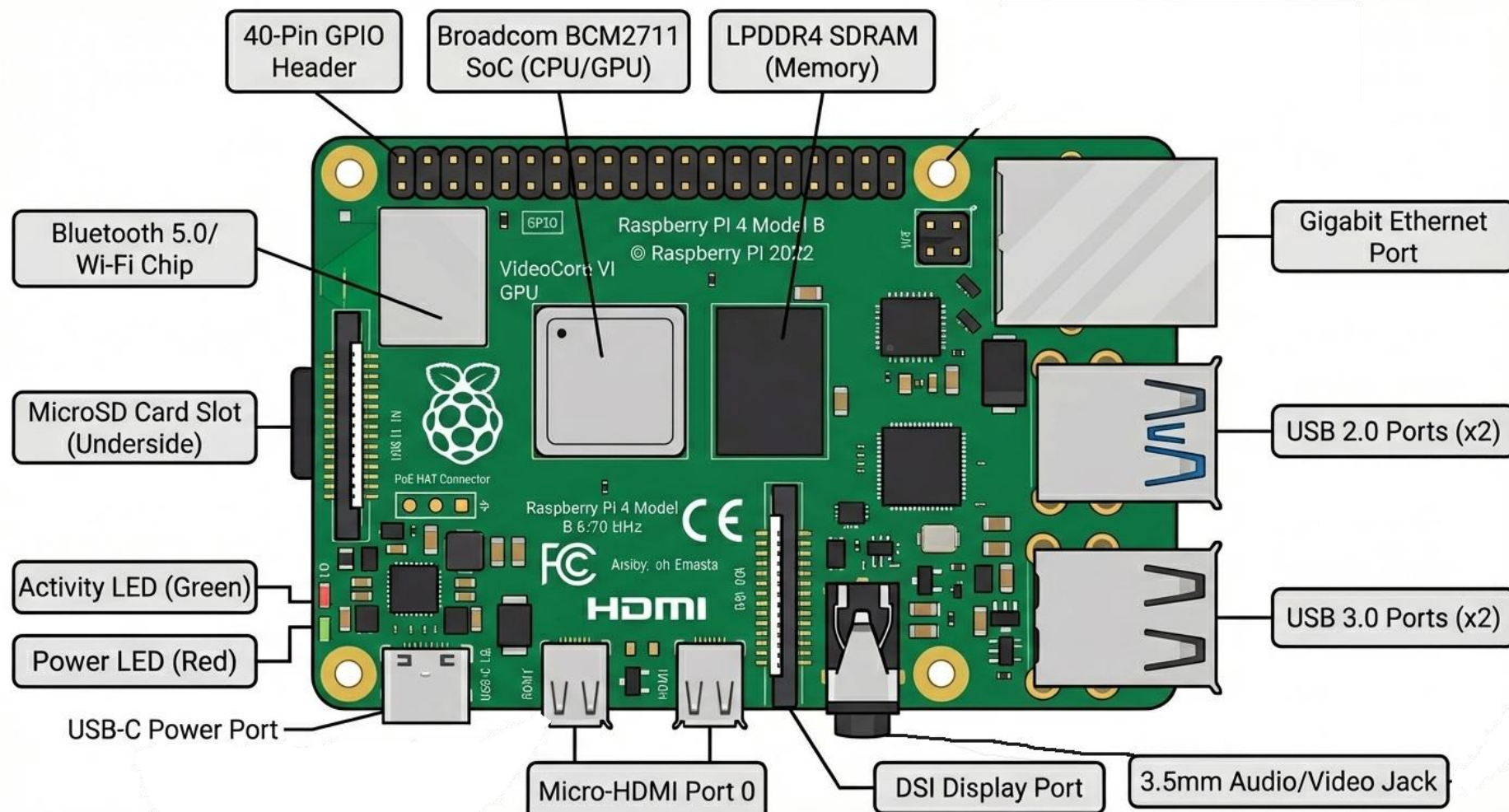
- **12. LEDs (Status Indicators)**

- Power LED
- Activity LED (SD card activity)

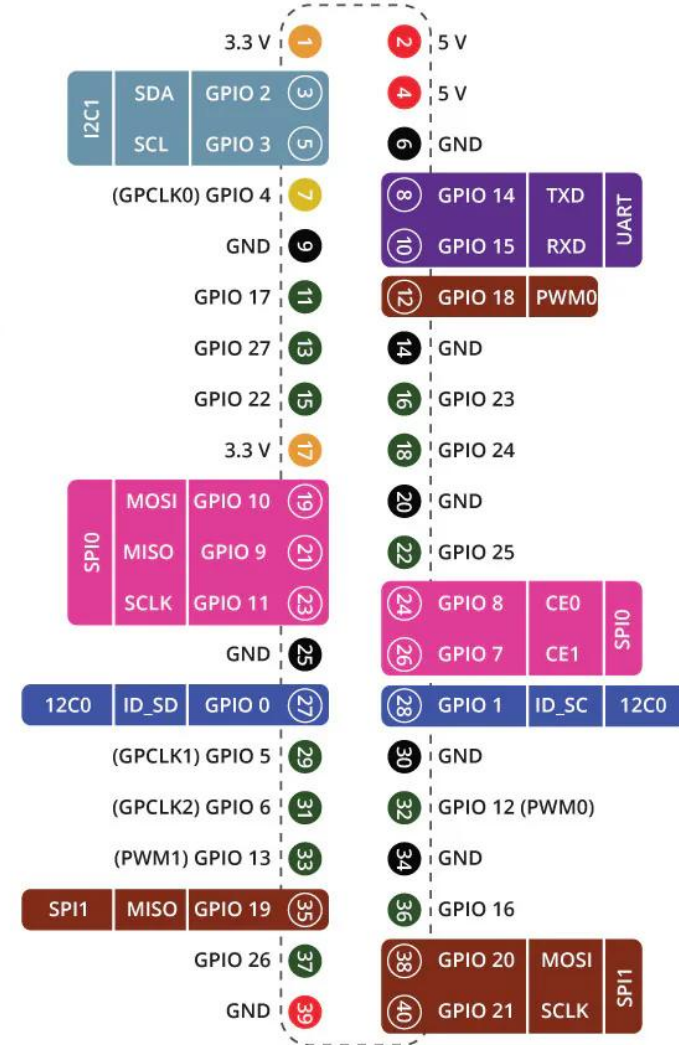
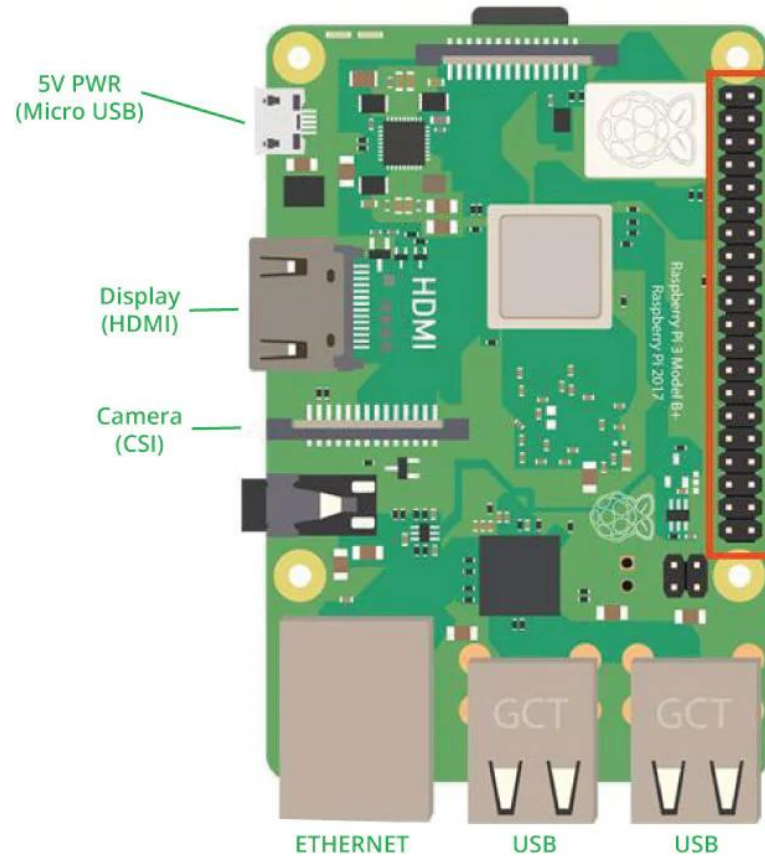
Raspberry Pi 1



Raspberry Pi 4



Raspberry Pi 4 GPIO Pins



GPIO Header (40 Pins) Raspberry Pi

1. Power Pins

Used to supply power to external devices:

5V Pins → Direct 5V supply

3.3V Pins → Regulated 3.3V supply

GND Pins → Ground reference

2. General Purpose I/O (Digital Pins)

Configurable as **Input** or **Output**

Used to:

- Read switches/sensors

- Control LEDs, relays, motors

Controlled via software (Python)

3. PWM (Pulse Width Modulation)

Used for:

- Motor speed control

- LED brightness control

- Servo control

Hardware PWM available on specific pins

4. I²C (Inter-Integrated Circuit)

Two-wire communication:

- SDA (Data)

- SCL (Clock)

Used for:

- Sensors

- RTC modules

- LCD displays

5. SPI (Serial Peripheral Interface)

High-speed synchronous communication

Pins include:

- MOSI

- MISO

- SCLK

- CE (Chip Enable)

Used for:

- ADC/DAC

- Displays

- Flash memory

GPIO Header (40 Pins) Raspberry Pi...

6. UART (Serial Communication)

TX (Transmit)

RX (Receive)

Used for:

- Serial debugging

- Communication with microcontrollers (Arduino, etc.)

7. Pin Numbering Methods

Physical numbering (1–40)

BCM numbering (Broadcom SoC GPIO number)

Interface Details

- **How I²C Works**

- Raspberry Pi acts as **Master**
- Sensor/Device acts as **Slave**
- Master generates clock (SCL)
- Data transferred bit-by-bit on SDA
- Each slave has a **unique 7-bit or 10-bit address**

What is SPI?

SPI (Serial Peripheral Interface) is a **synchronous, full-duplex communication protocol**.

- Master-slave architecture
- High-speed communication

What is UART?

UART (Universal Asynchronous Receiver Transmitter) is a serial, asynchronous communication protocol.

- No clock signal required
- Data transmitted bit-by-bit
- Uses predefined baud rate

Supported OS in Raspberry PI

OS Name	Base System	Type	Best Used For	Key Features
Raspberry Pi OS	Debian Linux	Official OS	Education, IoT, GPIO Projects	Optimized for Pi, Python preinstalled, Large community
Ubuntu	Ubuntu Linux	Desktop/Server	Cloud, AI, Server Applications	LTS support, Strong server ecosystem
Kali Linux	Debian Linux	Security OS	Penetration Testing, Cybersecurity	Pre-installed security tools
LibreELEC	Linux	Media OS	Home Theater / Media Center	Lightweight, Kodi-based
RetroPie	Debian-based	Gaming OS	Retro Gaming Console	Emulator support, Controller support
DietPi	Debian-based	Lightweight OS	Minimal IoT / Headless Systems	Very low resource usage

Raspberry Pi with Python

Python Library for PI

- **Usage:**
 - import RPi.GPIO as GPIO
 - import time
- **RPi.GPIO** : A Python library that allows Raspberry Pi GPIO pin control.
- **time Library:** Used for delays and timing control. Most used function:
 - time.sleep(seconds)

Functions in GPIO

- **GPIO.setmode():** Defines how pins are referenced.
- Ex: `GPIO.setmode(GPIO.BCM)`
 - **Two Modes:**
 - 1. GPIO.BCM**
 - Uses Broadcom SoC numbering.
 - Based on internal GPIO numbers.
 - Recommended for professional projects.
 - 2. GPIO.BOARD**
 - Uses physical pin numbers (1–40).
 - Easier for beginners.

Functions in GPIO...

- **GPIO.setup()**
 - Ex: GPIO.setup(18, GPIO.OUT)
- **Purpose:**
 - Configures a pin as:
 - GPIO.OUT → Output
 - GPIO.IN → Input

Functions in GPIO...

- **GPIO.output()**
 - GPIO.output(18, GPIO.HIGH)
 - GPIO.output(18, GPIO.LOW)
- **Purpose:**
 - Controls digital output.
 - GPIO.HIGH → 3.3V
 - GPIO.LOW → 0V

Functions in GPIO...

- **GPIO.input()**
 - GPIO.input(17)
- **Purpose:**
 - Reads digital input value.
 - Returns:
 - 1 (HIGH)
 - 0 (LOW)

Functions in GPIO...

- **GPIO.cleanup()**
 - GPIO.cleanup()
- **Purpose:**
 - Resets GPIO pins to default state.
- **Why Important?**
 - If not used:
 - Pins may remain HIGH
 - Can damage connected devices
 - Next program may behave unexpectedly
- **Best Practice:**
 - Use in try-finally block in professional code.

Example: LED Blinking Program

```
import RPi.GPIO as GPIO  
import time
```

```
GPIO.setmode(GPIO.BCM)  
GPIO.setup(18, GPIO.OUT)
```

```
while True:  
    GPIO.output(18, GPIO.HIGH)  
    time.sleep(1)  
    GPIO.output(18, GPIO.LOW)  
    time.sleep(1)
```

Input Example: Push Button

```
GPIO.setup(17, GPIO.IN)  
if GPIO.input(17) == GPIO.HIGH:  
    print("Button Pressed")
```

Headless Installation Raspberry Pi

**Headless Setup =
Running Raspberry Pi
without:**

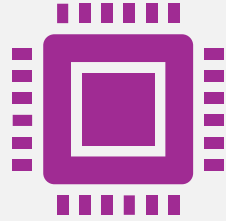
- Monitor
- Keyboard
- Mouse



Accessed remotely via:

- SSH
- PuTTY
- WinSCP

Requirements



- ✓ Raspberry Pi
- ✓ MicroSD Card (16GB+)
- ✓ Power Supply
- ✓ PC/Laptop
- ✓ Same WiFi network



Software needed on
PC:

Raspberry
Pi Imager
PuTTY
WinSCP

Flash OS with WiFi & SSH Enabled

- Insert SD card into PC
- Open Raspberry Pi Imager
- Choose Device
- Choose OS (Raspberry Pi OS)
- Click (Advanced Settings)
- Configure:
 - Hostname (e.g., raspberrypi)
 - Enable SSH
 - Set username & password
- Configure WiFi:
 - SSID
 - Password
 - Country: IN

Insert & Power ON

- Insert SD card into Pi
- Connect power
- Wait 1–2 minutes (first boot setup)
- Find Raspberry Pi IP Address or
- Use hostname directly:raspberrypi.local

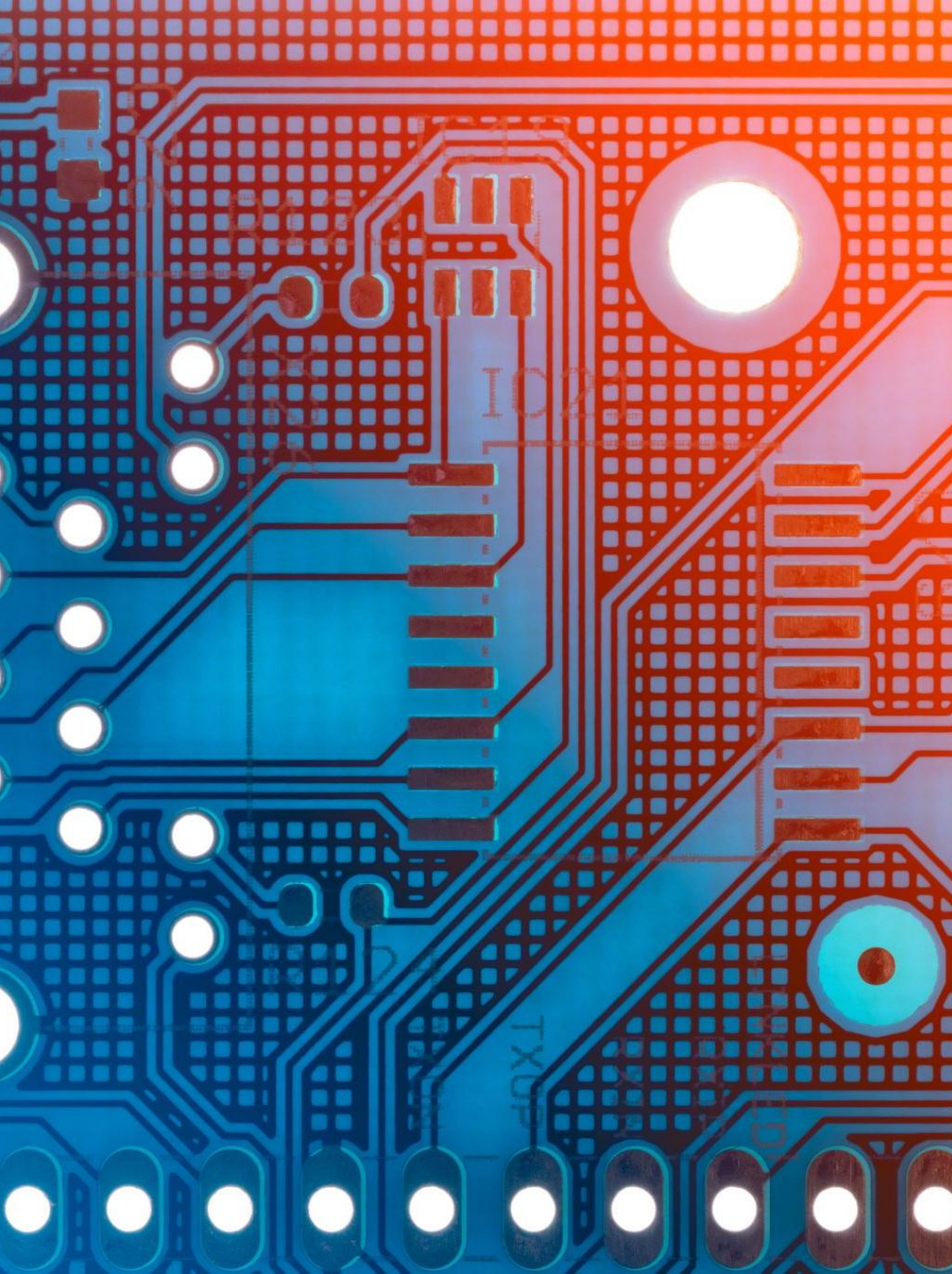
- Default Login Credentials
- If not changed during flashing:
- Username: pi
- Password: raspberry

Connect Using PuTTY

- Open PuTTY
- Enter IP Address
- Port: 22
- Connection Type: SSH
- Click Open
- Login:
 - login as: pi
 - password: raspberry
- Now you are inside Raspberry Pi terminal remotely.

Transfer Files Using WinSCP

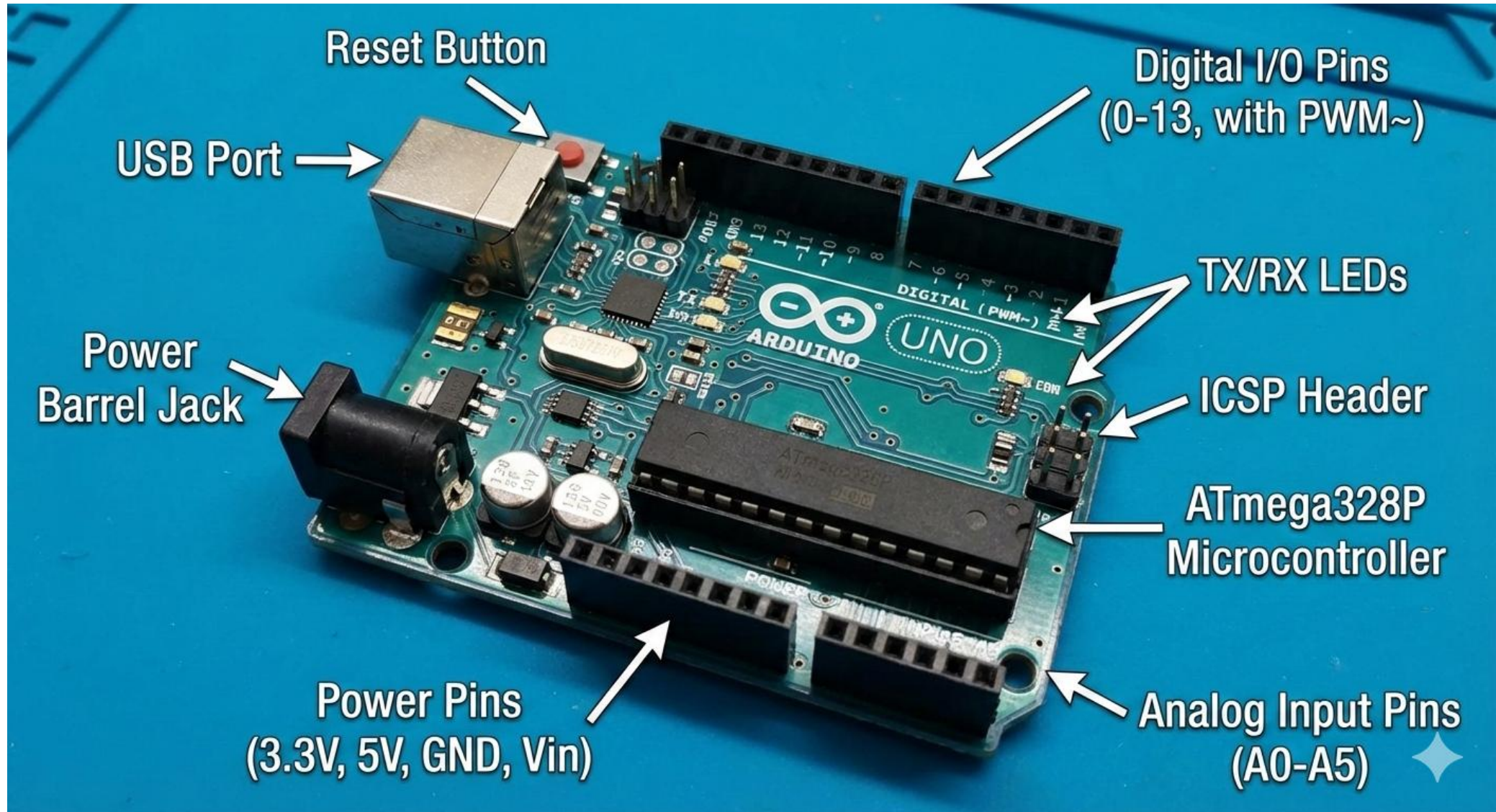
- Steps:
- Open WinSCP
- Hostname: Pi IP
- Username: pi
- Password: raspberry
- File protocol: SFTP
- Login
- You can:
- Upload Python files
- Download logs
- Transfer datasets



Introduction to Arduino

- **What is Arduino Uno?**
- Open-source microcontroller development board
- Based on ATmega328P
- Used for embedded systems & IoT projects
- Easy to program using Arduino IDE
- **Why Arduino?**
- Beginner-friendly
- Low cost
- Large community support
- Huge sensor/module ecosystem

Arduino



Arduino vs Raspberry Pi

Feature	Arduino	Raspberry Pi
Type	Microcontroller	Microcomputer
OS	No	Linux
Real-time	Yes	Not deterministic
Boot Time	Instant	~20 seconds
Power Consumption	Low	Higher

Arduino Uno Hardware Overview

- **ATmega328P Microcontroller:** The "brain" of the board that executes your code.
- **14 Digital I/O Pins:** Interface with components using binary on/off signals.
- **6 Analog Input Pins:** Read variable signals from sensors like knobs and light sensors.
- **USB Port:** Connects to your computer for programming and power.
- **Power Jack:** Supplies external power to the board when not connected to a computer.
- **Reset Button:** Restarts the currently uploaded program from the beginning.
- **Crystal Oscillator (16 MHz)::** Acts as a precise clock to synchronize all operations.

Arduino Uno Pin Description

- **Digital Pins (0–13)**
- Used for ON/OFF control
- 6 Pins support PWM (3,5,6,9,10,11)
- **Analog Pins (A0–A5)**
- 10-bit ADC (0–1023)
- Reads analog voltage (0–5V)
- **Power Pins**
- 5V
- 3.3V
- GND
- VIN

Digital Pins (0–13)

Digital Pins (0–13)

- Total **14 digital I/O pins** capable of reading or writing **HIGH (5V)** or **LOW (0V)** signals.
- Used to interface digital components such as:
 - LEDs
 - Switches
 - Relays
 - Digital sensors
- **PWM Support:**
 - Pins **3, 5, 6, 9, 10, 11** support **Pulse Width Modulation (PWM)** output.
 - PWM simulates analog output for controlling:
 - LED brightness
 - Motor speed
- Special functions:
 - **Pins 0 (RX) & 1 (TX):** Used for serial communication with PC or other devices.

Analog Input Pins (A0–A5)

Analog Input Pins (A0–A5)

- Total **6 analog inputs** that read continuous signals from 0–5V.
- Features a **10-bit ADC (Analog-to-Digital Converter)**:
 - Converts analog voltage to a numeric value between **0 and 1023**.
- Used for sensors that provide variable output such as:
 - Temperature sensors
 - Potentiometers
 - Light-dependent resistors (LDR)

Power Pins

Power Pins

These pins supply power to external modules and circuits:

5V Pin

- Provides regulated **5V output** from the onboard voltage regulator.
- Can power sensors, modules, and small circuits.

3.3V Pin

- Supplies **3.3V output** for low-voltage components.

GND (Ground)

- **Multiple GND pins** available for completing electric circuits.

VIN

- Used to power Arduino Uno using an external power supply (**7–12V input**).
- The power goes through the onboard regulator before reaching the board.

Arduino IDE

- Free software
- Write code (Sketch)
- Compile & Upload
- Serial Monitor for debugging
- Basic Structure of Arduino Program:

```
void setup() {  
}
```

```
void loop() {  
}
```

Program Structure Explained

setup()

- Runs once
- Used for initialization

loop()

- Runs continuously
- Main logic execution

pinMode() – Configure Pin Direction

- **Syntax:**
 - pinMode(pin, mode);
- **Parameters:**
 - **pin** → Arduino pin number (e.g., 2, 13, A0)
- **mode** →
 - INPUT
 - OUTPUT
 - INPUT_PULLUP
- **Purpose:**
 - Configures a pin as **input or output**.
- **Example:**
 - pinMode(13, OUTPUT);
 - pinMode(2, INPUT);
- **Why Important?**
 - Must be set inside setup()
 - Defines how the microcontroller treats the pin (read or write)

digitalRead() – Read Digital Signal

- **Syntax:**
 - `digitalRead(pin);`
- **Returns:**
 - HIGH (1) → 5V (or 3.3V depending on board)
 - LOW (0) → 0V
- **Used For:**
 - Reading button
 - Detecting ON/OFF signal
- **Example:**
 - `int buttonState = digitalRead(2);`
- **Works With:**
 - Pins configured as INPUT

digitalWrite() – Write Digital Signal

- **Syntax:**
 - digitalWrite(pin, value);
- **Value:**
 - HIGH
 - LOW
- **Used For:**
 - Turning LED ON/OFF
 - Controlling Relay
 - Sending digital signal
- **Example:**
 - digitalWrite(13, HIGH); // LED ON
 - digitalWrite(13, LOW); // LED OFF
- **Works With:**
 - Pins configured as OUTPUT

analogRead() – Read Analog Signal

- **Syntax:**
 - `analogRead(pin);`
- **Works With:**
 - Analog pins (A0, A1, A2...)
- **Returns:**
 - Integer value from **0 to 1023**
 - 10-bit ADC resolution (in boards like Arduino Uno)
- **Voltage Mapping (5V board):**
 - $0 \rightarrow 0V$
 - $1023 \rightarrow 5V$
- **Example:**
 - `int sensorValue = analogRead(A0);`
- **Used For:**
 - Temperature sensor
 - LDR
 - Potentiometer

analogWrite() – Generate PWM Signal

- **Syntax:**
 - `analogWrite(pin, value);`
- **Value Range:**
 - 0 → 255 (8-bit PWM)
- **Important:**
 - It is **not true analog output**
 - It generates **PWM (Pulse Width Modulation)**
- **Example:**
 - `analogWrite(9, 128); // 50% duty cycle`
- **Used For:**
 - LED brightness control
 - Motor speed control
- **WM Pins (Arduino Uno):**
 - 3, 5, 6, 9, 10, 11

Digital Output Example (Blink LED)

```
void setup() {  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);  
  delay(1000);  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```



Digital Input Example (Push Button)

```
void setup() {  
    pinMode(2, INPUT);  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    if(digitalRead(2) == HIGH) {  
        digitalWrite(13, HIGH);  
    } else {  
        digitalWrite(13, LOW);  
    }  
}
```



Analog Input Example

```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  int value = analogRead(A0);  
  Serial.println(value);  
  delay(500);  
}
```

PWM (Analog Output Simulation)

```
void setup() {  
  pinMode(9, OUTPUT);  
}  
  
void loop() {  
  analogWrite(9, 128);  
}
```

